RECURSION SECURITY

1. Preliminaries and Recursive Relations

This section fixes the notation used in the security analysis and restates, in the exact form the proofs consume, the objects introduced in the ZisK whitepaper: the arithmetization layer (AIRs, interactions and buses), the cryptographic components of the recursion (trace commitments, the multiset hash and challenge derivation), and the four relations attested by the nodes of the aggregation tree. This document compiles independently of the whitepaper, so nothing is assumed from there. Where a relation below differs from the whitepaper text, the difference is deliberate; all such differences are collected in Remark 1.2.

1.1. Notation and Parameters.

$\mathbb{F}$ is a prime field of order p (Goldilocks, $p = 2^{64} - 2^{32} + 1$) and $\mathbb{K}$ is its cubic extension, $|\mathbb{K}| = p^3$. $\mathbb{Z}_p^n$ denotes the additive group of n-vectors of integers modulo p, the same set as $\mathbb{F}^n$ viewed as a $\mathbb{Z}$-module. For integers $a \le b$, $[a, b]$ and $[a, b)$ are the closed and half-open integer intervals and $[b] := [1, b]$. Field elements are identified with integers in $[0, p)$ when needed. λ is the security parameter; "PPT" means probabilistic polynomial time in λ, and $\mathsf{negl}(\lambda)$ and $\mathsf{poly}(\lambda)$ denote negligible and polynomially bounded functions. All algorithms have oracle access to the random oracles introduced in Section 1.3; Q bounds the total number of oracle queries an adversary makes.

The recursion is governed by one integer parameter, the *leaf-count exponent* B: an aggregation tree may contain at most $2^B - 1$ leaves. We assume throughout that

$$2^{B+1} \le p, \quad 2^B < p, \quad 2^B \le \mathsf{poly}(\lambda) :$$

the first condition guarantees that sums of two range-checked counters never wrap around in $\mathbb{F}$, the second is the cardinality restriction required by the multiset hash, and the third makes the number of leaves, and hence the running time of the extractor, polynomial.

1.2. Arithmetization.

Definition 1.1 (AIR and Witness). An *algebraic intermediate representation* (AIR) of width W, depth D and V values is a finite set of *constraint polynomials*

$$\mathcal{A} \subset \mathbb{F}\Big[X_1^{(0)}, ..., X_W^{(0)}, ..., X_1^{(D)}, ..., X_W^{(D)}, Y_1, ..., Y_V\Big].$$

A *witness* of height N for $\mathcal{A}$ is a pair $w = (\boldsymbol{T}, \boldsymbol{a})$ with $\boldsymbol{T} \in \mathbb{F}^{N \times W}$, the *trace*, and $\boldsymbol{a} \in \mathbb{F}^V$, the *AIR values*. At row $r \in [N]$ the variable $X_j^{(k)}$ takes the value $T_j(r + k)$, with row indices taken modulo N, and Y_j takes the value a_j. The witness *satisfies* $\mathcal{A}$ if every constraint polynomial evaluates to zero at every row.

Definition 1.2 (Interaction). An *interaction* of an AIR $\mathcal{A}$ on the bus $b \in \mathbb{F}$ is a tuple $\iota = (b, m_\iota, \boldsymbol{e}_\iota)$, where the *multiplicity* m_ι and the *message* components $e_{\iota,1}, \ldots, e_{\iota,\ell_b}$ are polynomials over $\mathbb{F}$ in the variables of $\mathcal{A}$. Given a witness w and a row r, we write $m_\iota(w, r) \in \mathbb{F}$ and $\boldsymbol{e}_\iota(w, r) \in \mathbb{F}^{\ell_b}$ for their evaluations at row r. The set of bus identifiers in use is $\mathcal{B}$; every interaction on bus b has the common message length ℓ_b, and $\ell := \max_{b \in \mathcal{B}} \ell_b$.

Throughout, we fix a positive integer M and an indexed family of *AIR types*

$$\mathcal{C} = \{(\mathcal{A}_\tau, \mathcal{I}_\tau)\}_{\tau \in [M]},$$

where $\mathcal{A}_\tau$ is an AIR of width W_τ, depth D_τ and V_τ values, $\mathcal{I}_\tau$ is a finite set of interactions of $\mathcal{A}_\tau$. Given a fixed trace height N_τ, we write $\mathcal{W}_\tau := \mathbb{F}^{N_\tau \times W_\tau} \times \mathbb{F}^{V_\tau}$ for the witness space of type τ and fix the public-statement space $\mathcal{P} := \mathbb{F}^k$.

Definition 1.3 (Local Satisfaction). For $\tau \in [M]$, $\boldsymbol{p} \in \mathcal{P}$ and $w = (\boldsymbol{T}, \boldsymbol{a}) \in \mathcal{W}_\tau$, the predicate $\mathrm{Local}_\tau(\boldsymbol{p}, w)$ holds if and only if

1. w satisfies $\mathcal{A}_\tau$.
2. The fixed columns of $\boldsymbol{T}$ equal the fixed data prescribed by $\mathcal{A}_\tau$ (cached columns, committed once at setup, are treated as fixed data whose commitment is part of the verification key of type τ).
3. $\mathrm{Bind}_\tau(\boldsymbol{p}, w)$: every trace cell and AIR value that $\mathcal{A}_\tau$ designates as a binding to the public statement equals the designated coordinate of $\boldsymbol{p}$.

The program being executed is not a separate parameter: the program ROM and the boot state are bound through fixed and cached columns of the relevant AIR types, hence through their verification keys. This plays the role of the program code hard-wired into the step circuit of the reference architecture.

Definition 1.4 (Message multiset of a witness). Let $\mathcal{V} := \left\{(b, v) : b \in \mathcal{B}, v \in \mathbb{F}^{\ell_b}\right\}$ be the set of *bus messages*. An *$\mathbb{F}$-weighted multiset* of messages is a finitely supported map $D : \mathcal{V} \to \mathbb{F}$; we write $|D|$ for the size of its support and add such maps pointwise. For $\tau \in [M]$ and $w \in \mathcal{W}_\tau$, the *message multiset* of w is

$$\mathrm{Msg}_\tau(w)(b, v) := \sum_{\iota \in \mathcal{I}_\tau, b_\iota = b} \sum_{r=1}^{N_\tau} m_\iota(w, r) \cdot \mathbf{1}[\boldsymbol{e}_\iota(w, r) = v].$$

Its support has at most $N_\tau \cdot |\mathcal{I}_\tau|$ elements.

A row that sends a message with positive multiplicity and a row of the same witness that receives it with negative multiplicity cancel in $\mathrm{Msg}_\tau(w)$: the multiset records the net flow of every message out of the witness, and nothing about the mechanism by which that flow is checked.

Definition 1.5 (Bus fingerprint). A *bus fingerprint* is a family $\mathcal{F} = \left\{\mathcal{F}_\chi\right\}_{\chi \in \mathbb{K}^2}$ of maps from $\mathbb{F}$-weighted multisets of messages to $\mathbb{K} \cup \{\perp\}$ such that

1. (*additivity*) $\mathcal{F}_\chi(0) = 0$, and $\mathcal{F}_\chi(D + D') = \mathcal{F}_\chi(D) + \mathcal{F}_\chi(D')$ whenever the right-hand side is defined;
2. (*soundness*) there is a non-decreasing function $\varepsilon_{\mathcal{F}}$ such that for every $D \neq 0$ with $|D| \leq \Lambda$ and uniform $\chi \in \mathbb{K}^2$, $\Pr_\chi\big[\mathcal{F}_\chi(D) = 0\big] \leq \varepsilon_{\mathcal{F}}(\Lambda)$.

The *bus contribution* of $w \in \mathcal{W}_\tau$ under the *challenge pair* $\chi \in \mathbb{K}^2$ is

$$\mathrm{BusCont}_\chi(\mathcal{I}_\tau, w) := \mathcal{F}_\chi(\mathrm{Msg}_\tau(w)) \in \mathbb{K} \cup \{\bot\}.$$

A fingerprint is a keyed, additive summary of a multiset that vanishes on a nonzero multiset only at few keys; it is to bus balancing what the multiset hash $\mathcal{H}$ of Section 1.3 is to challenge derivation. ZisK instantiates it with LogUp [Hab22]: for $\chi = (\alpha, \gamma)$ the message (b, v) is *compressed* to $\hat{e}_\gamma(b, v) := b + \sum_{j=1}^{\ell_b} \gamma^j v_j \in \mathbb{K}$ and

$$\mathcal{F}^{\mathrm{LogUp}}_\chi(D) := \sum_{(b,v) \in \,\mathrm{supp}\, D} \frac{D(b, v)}{\alpha + \hat{e}_\gamma(b, v)},$$

with $\mathcal{F}^{\mathrm{LogUp}}_\chi(D) := \bot$ if some denominator vanishes. Lemma 2.9 shows that this is a bus fingerprint with $\varepsilon_{\mathcal{F}}(\Lambda) = \Big(\ell \cdot \binom{\Lambda}{2} + \Lambda - 1\Big)/|\mathbb{K}|$. The AIR argument does not evaluate this closed form. It constrains a running sum over the rows of the trace, adding $m_\iota(w, r)/\big(\alpha + \hat{e}_\gamma(b_\iota, \boldsymbol{e}_\iota(w, r))\big)$ at every row through an explicitly constrained inverse, and exposes the final value s; this is the whitepaper's $S(N_\tau)$. Grouping the row terms by message shows that the exposed value equals $\mathcal{F}^{\mathrm{LogUp}}_\chi(\mathrm{Msg}_\tau(w))$ whenever every row denominator is nonzero. The circuit therefore proves a sub-relation of $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$ below: it additionally rejects when a row of zero multiplicity, or a pair of rows that cancel in the multiset, hits a pole. This only strengthens the statements of Section 2, since knowledge soundness for a sub-relation implies knowledge soundness for the relation.

Definition 1.6 (Typed witness collection). A *typed witness collection* is a finite list

$$L = \Big((\tau_1, w_1), \ldots, \big(\tau_{|L|}, w_{|L|}\big)\Big), \quad \tau_i \in [M], \quad w_i \in \mathcal{W}_{\tau_i}.$$

Types and witnesses may repeat, and each entry counts separately; the order of the list carries no meaning. The *number of bus terms* of L is $\Lambda(L) := \sum_{i=1}^{|L|} N_{\tau_i} \cdot \big|\mathcal{I}_{\tau_i}\big|$, and we set $\Lambda_{\max} := (2^B - 1) \cdot \max_{\tau \in [M]} N_\tau |\mathcal{I}_\tau|$, so that $\Lambda(L) \leq \Lambda_{\max}$ whenever $|L| < 2^B$.

Definition 1.7 (Balance of a witness collection). The *signed multiplicity* of a typed witness collection L is the $\mathbb{F}$-weighted multiset

$$\mathcal{M}_L := \sum_{i=1}^{|L|} \mathrm{Msg}_{\tau_i}(w_i),$$

that is,

$$\mathcal{M}_L(b,v) = \sum_{i=1}^{|L|} \sum_{\iota \in \mathcal{I}_{\tau_i}, b_\iota = b} \sum_{r=1}^{N_{\tau_i}} m_\iota(w_i, r) \cdot \mathbf{1}[\boldsymbol{e}_\iota(w_i, r) = v].$$

The collection is *balanced*, written $\mathrm{Balanced}(L)$, if $\mathcal{M}_L = 0$, i.e. $\mathcal{M}_L(b,v) = 0$ for every bus $b \in \mathcal{B}$ and every message $v \in \mathbb{F}^{\ell_b}$. Note that $|\mathcal{M}_L| \le \Lambda(L)$.

This is the whitepaper's bus-balancing condition extended from one witness per AIR type to an arbitrary collection of instances. Its multiplicities are field elements: $\mathrm{Balanced}(L)$ does not by itself assert that every message is sent and received the same number of times as integers, since p copies of a message cancel. We return to this point in Remark 2.4.

1.3. **Proof-System Components.**

A *non-interactive argument system* for a relation $\mathcal{R}$ is a pair $\Pi = (\mathsf{Prove}, \mathsf{Verify})$ with a verification key vk_Π produced at setup; $\mathsf{Verify}(\mathsf{vk}_\Pi, x, \pi) \in \{\mathrm{accept}, \mathrm{reject}\}$ is deterministic. We write $V_X(x,\pi) := \mathsf{Verify}(\mathsf{vk}_X, x, \pi)$ for the verifier of the system attesting to the relation $\mathcal{R}_X$, and simply $V(\mathsf{vk}, x, \pi)$ when the key is a variable. The recursion uses the following components.

- **Trace commitment.** Com is a deterministic commitment scheme with algorithm $\mathsf{Commit} : \bigcup_\tau \mathcal{W}_\tau \to \{0,1\}^*$, built from a hash function modelled as a random oracle (a Merkle tree in ZisK). A commitment $\mathrm{com} = \mathsf{Commit}(\boldsymbol{T}, \boldsymbol{a})$ covers every column of the trace and every AIR value; the auxiliary columns that the prover of an AIR argument adds for the LogUp running sums are internal to that argument and are not part of the witness. The argument system for an AIR type takes the challenge pair χ as a public input, so the witness is challenge-independent.
- **Hash-to-group and multiset hash.** $h : [M] \times \{0,1\}^* \to \mathbb{Z}_p^n$ maps a *typed commitment* (τ, com) to a group element, and
$$\mathcal{H}(\mathcal{S}) := \sum_{x \in \mathcal{S}} \mathrm{mult}_{\mathcal{S}}(x) \cdot h(x) \in \mathbb{Z}_p^n$$
is the induced multiset hash on finite multisets $\mathcal{S}$ of typed commitments, where $\mathrm{mult}_{\mathcal{S}}(x)$ is the multiplicity of x in $\mathcal{S}$. $\mathcal{H}$ is additively homomorphic over multiset union, which is what lets the aggregation tree accumulate it. ZisK instantiates h and $\mathcal{H}$ with LtHash [Cla+03, Zis25], $n = 372$; the inner digest of h is a Poseidon2 hash [GKS23] of the commitment.
- **Challenge derivation.** $\mathsf{DeriveAlpha} : \mathbb{Z}_p^n \to \mathbb{K}^2$ maps an accumulated group element to a challenge pair $\chi = (\alpha, \gamma)$. ZisK derives α as a Poseidon2 hash of the accumulator and γ as a hash of α; we treat the pair as a single oracle output, see Remark 1.2.
- **Argument systems.** For every type $\tau \in [M]$ an argument system $\Pi_{\mathrm{AIR}}^{(\tau)}$ for $\mathcal{R}_{\mathrm{AIR}}^{(\tau)}$ and one $\Pi_{\mathrm{LEAF}}^{(\tau)}$ for $\mathcal{R}_{\mathrm{LEAF}}^{(\tau)}$; a single Π_{AGG} for $\mathcal{R}_{\mathrm{AGG}}$; and a single Π_{FINAL} for $\mathcal{R}_{\mathrm{FINAL}}$. The relations are defined in Section 1.4.

The verification keys involved play different roles, which the whitepaper distinguishes only in prose; Table 1 records them.

Key	**Selected by**	**Hard-coded in**
$\mathsf{vk}_{\mathrm{AIR}}^{(\tau)}$	the type τ of the leaf verifier circuit	$\Pi_{\mathrm{LEAF}}^{(\tau)}$
$\mathsf{vk}_{\mathrm{LEAF}}^{(\tau)}$	the child type $\tau_X \in [M]$ read from the child statement	Π_{AGG} (all M keys)
$\mathsf{vk}_{\mathrm{AGG}}$	the child type $\tau_X = M+1$; the key itself arrives as the public vk	Π_{FINAL} only
$\mathsf{vk}_{\mathrm{FINAL}}$	—	the external verifier

TABLE 1. Verification-key ledger. The circuit type τ selects which hard-coded key an aggregation node uses for a child; the propagated public vk is the aggregation verifier's own key, which no aggregation circuit can hard-code and which the Final Verifier fixes.

Definition 1.8 (Recursive dispatch). For a type $\tau \in [M+1]$ and a key vk, the *dispatched verifier* is

$$V_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}](x,\pi) := \begin{cases} V_{\mathrm{LEAF}}^{(\tau)}(x,\pi) & \text{if } \tau \in [M], \\ V(\mathsf{vk},x,\pi) & \text{if } \tau = M+1, \end{cases}$$

and $V_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}]$ rejects everything if $\tau \notin [M+1]$. The *typing predicate* on a type and a counter is

$$\mathsf{Typed}(\tau,\kappa) :\Leftrightarrow (\tau \in [M] \wedge \kappa = 1) \vee (\tau = M+1 \wedge 2 \le \kappa < 2^B).$$

The dispatch is a function of the key vk carried in the statement, not of any fixed key. This is what removes the apparent circularity of a circuit verifying proofs of itself: $\mathcal{R}_{\mathrm{AGG}}$ below is one ordinary relation over statements that contain an arbitrary string vk; the key $\mathsf{vk}_{\mathrm{AGG}}$ of the argument system for that relation is produced afterwards at setup; and $\mathcal{R}_{\mathrm{FINAL}}$ fixes $\mathsf{vk} := \mathsf{vk}_{\mathrm{AGG}}$, at which point $V_{\mathrm{REC}}^{(M+1)}[\mathsf{vk}_{\mathrm{AGG}}] = V_{\mathrm{AGG}}$.

1.4. Recursive Relations.

We write a relation as $\mathcal{R} = \{(x;w) \mid \varphi(x,w)\}$ with public statement x and witness w. The *recursive statement* carried by every node of the tree is the tuple $\sigma = (s,\tau,\kappa,\boldsymbol{c})$ of a bus partial sum $s \in \mathbb{K}$, a type $\tau \in [M+1]$, a leaf counter $\kappa \in \mathbb{F}$ and a challenge accumulator $\boldsymbol{c} \in \mathbb{Z}_p^n$; we access its components as $\sigma.s$, $\sigma.\tau$ and so on.

Definition 1.9 (AIR relation). For $\tau \in [M]$,

$$\mathcal{R}_{\mathrm{AIR}}^{(\tau)} = \left\{ (\boldsymbol{p}, \chi, s, \mathrm{com}); (\boldsymbol{T}, \boldsymbol{a}) \;\middle|\; \begin{array}{ll} & \mathrm{Local}_\tau(\boldsymbol{p}, (\boldsymbol{T}, \boldsymbol{a})) \\ \wedge & s = \mathrm{BusCont}_\chi(\mathcal{I}_\tau, (\boldsymbol{T}, \boldsymbol{a})) \\ \wedge & \mathsf{Commit}(\boldsymbol{T}, \boldsymbol{a}) = \mathrm{com} \end{array} \right\}$$

with $\boldsymbol{p} \in \mathcal{P}$, $\chi \in \mathbb{K}^2$, $s \in \mathbb{K}$ and $(\boldsymbol{T}, \boldsymbol{a}) \in \mathcal{W}_\tau$.

The relation is in *commit-and-prove* form: the trace commitment is part of the statement. A STARK proof for $\mathcal{A}_\tau$ already contains this commitment, so $V_{\mathrm{AIR}}^{(\tau)}$ simply reads it off the proof and compares it with the statement component; the point of exposing it is that the leaf verifier can then bind the challenge contribution to a value that is a function of the statement and witness rather than of an object that occurs in neither (Remark 1.2).

Definition 1.10 (Leaf-verifier relation). For $\tau \in [M]$,

$$\mathcal{R}_{\mathrm{LEAF}}^{(\tau)} = \left\{ (\boldsymbol{p}, \chi, \sigma, \mathsf{vk}); (\mathrm{com}, \pi_{\mathrm{AIR}}) \;\middle|\; \begin{array}{ll} & \sigma = (s, \tau, 1, \boldsymbol{c}) \\ \wedge & V_{\mathrm{AIR}}^{(\tau)}((\boldsymbol{p}, \chi, s, \mathrm{com}), \pi_{\mathrm{AIR}}) = \mathrm{accept} \\ \wedge & \boldsymbol{c} = h(\tau, \mathrm{com}) \end{array} \right\}$$

The key vk is carried in the statement but not used.

Definition 1.11 (Aggregation relation). Write $x_X := (\boldsymbol{p}, \chi, \sigma_X, \mathsf{vk})$ for the statement of the child $X \in \{L, R\}$ and $\sigma_X = (s_X, \tau_X, \kappa_X, \boldsymbol{c}_X)$ for its recursive statement. Then

$$\mathcal{R}_{\mathrm{AGG}} = \left\{ (\boldsymbol{p}, \chi, \sigma, \mathsf{vk}); ((\sigma_L, \pi_L), (\sigma_R, \pi_R)) \;\middle|\; \begin{array}{ll} & \sigma = (s, M+1, \kappa, \boldsymbol{c}) \\ \wedge & \mathsf{Typed}(\tau_X, \kappa_X) \text{ for } X \in \{L, R\} \\ \wedge & V_{\mathrm{REC}}^{(\tau_X)}[\mathsf{vk}](x_X, \pi_X) = \mathrm{accept} \text{ for } X \in \{L, R\} \\ \wedge & \kappa = \kappa_L + \kappa_R \text{ and } s = s_L + s_R \text{ and } \boldsymbol{c} = \boldsymbol{c}_L + \boldsymbol{c}_R \end{array} \right\}$$

Two invariants of this relation are used repeatedly. First, the *propagation invariant*: both children are verified on statements that carry the parent's own $(\boldsymbol{p}, \chi, \mathsf{vk})$, so every node of a valid tree carries the root's public statement, challenge pair and key; in particular an aggregation child is verified under $V(\mathsf{vk}, \cdot, \cdot)$ for the very vk the parent declares. Second, the *counter invariant*: Typed places both children's counters in $[1, 2^B)$, and $2^{B+1} \le p$ makes $\kappa = \kappa_L + \kappa_R$ an identity of integers, so every child has $\kappa_X \le \kappa - 1$.

Definition 1.12 (Final relation). Fix the aggregation key $\mathsf{vk}_{\mathrm{AGG}}$ as a parameter. Then

$$\mathcal{R}_{\mathrm{FINAL}} = \left\{ \boldsymbol{p}; (\chi, \sigma, \pi) \;\middle|\; \begin{array}{ll} & \sigma = (s, \tau, \kappa, \boldsymbol{c}) \text{ and } \mathsf{Typed}(\tau, \kappa) \\ \wedge & V_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}_{\mathrm{AGG}}), \pi) = \mathrm{accept} \\ \wedge & s = 0 \\ \wedge & \chi = \mathsf{DeriveAlpha}(\boldsymbol{c}) \end{array} \right\}$$

Only the public statement $\boldsymbol{p}$ survives at the root: the challenge pair, the recursive statement and the child proof are witnesses. Dispatching on τ lets the Final Verifier accept a leaf-verifier proof directly when the execution fits in a single AIR instance, the degenerate case treated as a remark in the whitepaper.

Remark 1.1 (Well-foundedness). *With the whitepaper's range check $\kappa_L, \kappa_R < 2^B$ alone, a child with counter 0 is admissible. Such a child cannot be a leaf (leaves have counter 1), so it is an aggregation node whose children again have counter 0, and so on: the relation admits infinite descending chains of locally satisfiable aggregation statements, none of which contains a leaf, while every node of the chain contributes an arbitrary $(s, \boldsymbol{c})$ to the sums. No extractor whose work is bounded by the root statement can exist for such a relation, and no knowledge-soundness break is ever charged. The typing predicate of Definition 1.8 removes the problem: by the counter invariant, every child has a strictly smaller counter, a valid tree with root counter κ has exactly κ leaves and $\kappa - 1$ aggregation nodes, and strong induction on κ is well-founded. This is the analogue of the step-count typing of the reference architecture's combine circuit.*

Remark 1.2 (Deviations from the whitepaper). *The relations above differ from the whitepaper's in the following respects, each needed by the proofs of Section 2.*

1. *$\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$ exposes the trace commitment in its statement, and $\mathcal{R}^{(\tau)}_{\mathrm{LEAF}}$ takes it as a witness component. In the whitepaper the leaf relation computes $\boldsymbol{c} = h(\mathrm{com}(\boldsymbol{T}, \boldsymbol{a}))$ although neither $\boldsymbol{T}$ nor $\boldsymbol{a}$ is a variable of that relation.*
2. *The challenge contribution is $h(\tau, \mathrm{com})$ rather than $h(\mathrm{com})$. A commitment determines the witness but not the type under which it is proved; without the type, a prover could choose the type, and hence the bus contribution, after the challenge is known (Proposition 2.12).*
3. *$\mathcal{R}_{\mathrm{AGG}}$ is parametrized by the public vk through the dispatched verifier, the types of both children are constrained to $[M + 1]$, and both children are typed by Typed; in particular $\kappa_X \geq 1$ (Remark 1.1). The whitepaper states the same key-propagation rule in prose and range-checks the counters only from above.*
4. *$\mathcal{R}^{(\tau)}_{\mathrm{LEAF}}$ requires $\sigma.\tau = \tau$, so that the type declared in the statement is the type of the circuit.*
5. *$\mathcal{R}_{\mathrm{FINAL}}$ dispatches on the child type and requires $\mathsf{Typed}(\tau, \kappa)$, so $\kappa \geq 1$ and the single-leaf case is covered without a separate remark.*
6. *The leaves receive the challenge pair $\chi = (\alpha, \gamma)$ and the Final Verifier checks $\chi = \mathsf{DeriveAlpha}(\boldsymbol{c})$. ZisK broadcasts only α and derives γ from it by a further hash; modelling that hash as a random oracle as well, the analysis of Section 2 goes through with an additional $(Q + 1)/|\mathbb{K}|$ term for the event that a derived α collides with a previously queried input, which we suppress.*
7. *The bus contribution is defined abstractly, as a bus fingerprint of the witness's message multiset (Definition 1.5), with LogUp as the instantiation*

(Lemma 2.9); the whitepaper uses the name $\mathrm{BusCont}$ *for the LogUp running sum without defining it. The leaf-count exponent* B *is used consistently (Section 1).*

2. Security

2.1. **Overview.**

We prove that the ZisK aggregation tree is *correct-witness extractable*: any adversary that produces an accepting Final proof for a public statement $\boldsymbol{p}$ can be used to extract, in a straight-line fashion, a typed witness collection L such that every witness satisfies its AIR and is consistent with $\boldsymbol{p}$, and every bus balances across the whole collection. The proof descends the aggregation tree of Section 1 layer by layer, reducing everything to the knowledge soundness of the four kinds of argument systems ($\Pi^{(\tau)}_{\mathrm{AIR}}$, $\Pi^{(\tau)}_{\mathrm{LEAF}}$, Π_{AGG} and Π_{FINAL}), the straight-line extractability of the trace commitment Com, the multiset collision resistance of $\mathcal{H}$, and the soundness of the bus fingerprint, instantiated by LogUp, at a random-oracle challenge.

Remark 2.1 (Idealized proof model). *The analysis is idealized in three respects.*

First, Definition 2.1 postulates a straight-line extractor that runs in essentially the time of the adversary. Because the argument systems are hash-based, the natural model is the random oracle model (ROM), in which such extractors exist for a single argument [BCS16]. Composing straight-line extraction across the layers of a recursion tree, however, requires every argument to remain knowledge-sound relative to the same oracle, and such relativized succinct arguments do not exist in general [BCG24]. Like the reference architecture, we idealize over this obstruction and assume that straight-line extraction composes.

Second, the bus argument is sound only if the collection of witnesses that is hashed into the accumulator $\boldsymbol{c}$ *is fixed before the random oracle answers at* $\boldsymbol{c}$*. For the trace commitments this follows from straight-line extractability of Merkle trees in the ROM. For the algebraic accumulator* $\mathcal{H}$ *it does not: distinct multisets with equal hash exist in abundance (the group* $\mathbb{Z}_p^n$ *is far smaller than the set of multisets of fewer than* 2^B *commitments), only the hardness of finding them protects the scheme, and recovering a multiset from its accumulator is a subset-sum problem. We therefore charge the event that two admissible multisets known to the adversary share an accumulator to the collision resistance of* $\mathcal{H}$*, although a straight-line reduction cannot in general exhibit the second multiset; Remark 2.3 discusses the rigorous alternative.*

Third, while the analysis produces explicit advantage bounds and reduction running times (Theorem 2.21), the two idealizations mean that these should be read as statements about the reduction structure (the recursion topology is sound and the relations compose) rather than as a concrete security level.

Remark (Scope). *The target relation of this section speaks about AIR witnesses, not about guest executions: it is the recursion layer of the chain that leads from an*

accepting final proof to a correct execution of the guest program. Not covered here, and left as named obligations, are the semantic layer (that a balanced collection of locally satisfying witnesses encodes a single execution of the committed program, including the memory argument and continuation stitching), the knowledge soundness of the individual STARK arguments (a property of the backend, quantified by their own analysis), completeness, zero knowledge, and any wrapper or compression layers placed above the Final Verifier, each of which adds one more lemma of the kind proved for the leaf-verifier layer.

2.2. **Definitions.**

Definition 2.1 (Knowledge soundness). A non-interactive argument system $\Pi = (\mathsf{Prove}, \mathsf{Verify})$ for a relation $\mathcal{R}$ is *knowledge-sound* if there exists a PPT algorithm $\mathcal{E}$, the *extractor*, such that for every PPT adversary $\mathcal{A}$

$$\mathsf{Adv}^{\mathsf{ks}}_{\Pi}(\mathcal{A}) := \Pr\big[(x,\pi) \leftarrow \mathcal{A}\big(1^{\lambda}\big); w \leftarrow \mathcal{E}(x,\pi) : \\ \mathsf{Verify}(\mathsf{vk}_{\Pi}, x, \pi) = \mathrm{accept} \wedge (x;w) \notin \mathcal{R}\big] \leq \mathsf{negl}(\lambda),$$

the probability taken over the coins of $\mathcal{A}$ and $\mathcal{E}$ and the random oracles, to which all algorithms have access. We assume that the running time of $\mathcal{E}(x,\pi)$ is at most $\mathsf{Time}\big(\mathcal{A}\big(1^{\lambda}\big)\big) + \mathsf{poly}(\lambda)$ for every $\mathcal{A}$ and every (x,π) in the support of $\mathcal{A}$'s output: the extractor incurs an additive polynomial overhead over one evaluation of $\mathcal{A}$. We call $\mathsf{Adv}^{\mathsf{ks}}_{\Pi}(\mathcal{A})$ the *knowledge-soundness error* of $\mathcal{A}$ against Π.

Remark (Straight-line extraction). *The extractor of Definition 2.1 is straight-line: it receives only the statement and the proof and does not rewind the adversary. For succinct proofs this is not achievable in the plain model, and in the ROM it holds for one argument but does not compose across a recursion (Remark 2.1). The definition is a proxy that makes the reduction structure explicit; a concrete instantiation replaces it by ROM extractability with the overhead discussed there.*

Definition 2.2 (Straight-line extractable commitment). The trace commitment Com is *straight-line extractable* if there is a PPT algorithm $\mathcal{E}_{\mathsf{Com}}$ that, given a transcript $\mathcal{T}$ of random-oracle queries and answers and a value com, outputs a witness or $\perp$, such that for every PPT adversary $\mathcal{A}$ that at some point during its execution announces a value com and at the end outputs a witness w,

$$\mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{A}) := \Pr\big[\mathsf{Commit}(w) = \mathrm{com} \wedge \mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{\leq t}, \mathrm{com}) \neq w\big] \leq \mathsf{negl}(\lambda),$$

where t is the time of the announcement and $\mathcal{T}_{\leq t}$ the transcript up to it.

For a Merkle tree over a random oracle the extractor recomputes the leaves from the tree of queried nodes below the root; it fails only if the adversary later opens the root to a leaf it had not queried, which requires guessing an oracle output or finding a collision, so $\mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{A}) \leq (Q^2+Q)/|\mathrm{Range}|$ for the range of the oracle. Straight-line extractability implies binding: two openings of one commitment would both have to equal the extractor's output.

Definition 2.3 (Multiset collision resistance). The multiset hash $\mathcal{H}$ is *collision-resistant on multisets of cardinality below* 2^B if for every PPT adversary $\mathcal{A}$

$$\mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}(\mathcal{A}) := \Pr\big[(\mathcal{S},\mathcal{S}') \leftarrow \mathcal{A}\big(1^\lambda\big) : \mathcal{S} \neq \mathcal{S}' \wedge |\mathcal{S}|, |\mathcal{S}'| < 2^B \wedge \mathcal{H}(\mathcal{S}) = \mathcal{H}(\mathcal{S}')\big] \leq \mathsf{negl}(\lambda).$$

The cardinality restriction is necessary: $\mathbb{Z}_p^n$ has exponent p, so adding p copies of any element to a multiset leaves $\mathcal{H}$ unchanged, and $2^B < p$ rules this out. For LtHash, collision resistance on bounded multisets reduces to the short-integer-solution problem [Ajt96, Zis25]; the reduction requires the inner digest of h to be collision resistant on its own, which a single field element is not, so we assume the digest has at least 2λ bits (Section 2.6).

Definition 2.4 (Random-oracle model). The hash-to-group function h, the challenge derivation $\mathsf{DeriveAlpha}$ and the hash function underlying Com are independent random oracles. An adversary makes at most Q queries in total; every algorithm, including verifiers and extractors, has oracle access.

Definition 2.5 (ZisK proof system). A *ZisK proof system* is a tuple

$$\mathsf{ZisK} = \left(\mathcal{C}, \mathsf{Com}, h, \mathcal{H}, \mathsf{DeriveAlpha}, \mathcal{F}, \left\{\Pi^{(\tau)}_{\mathrm{AIR}}\right\}_{\tau\in[M]}, \left\{\Pi^{(\tau)}_{\mathrm{LEAF}}\right\}_{\tau\in[M]}, \Pi_{\mathrm{AGG}}, \Pi_{\mathrm{FINAL}}\right)$$

consisting of the AIR-type family $\mathcal{C}$, the components of Section 1.3 and the bus fingerprint $\mathcal{F}$ (Definition 1.5), where the argument systems attest to the relations of Section 1.4, the key hard-coded by Π_{FINAL} is the key of Π_{AGG}, and the parameters satisfy the standing assumptions of Section 1.

Definition 2.6 (Bounded global satisfiability). The *target relation* $\mathcal{R}_{\mathsf{ZisK}}$ has public statements $\boldsymbol{p} \in \mathcal{P}$ and typed witness collections L as witnesses:

$$(\boldsymbol{p}; L) \in \mathcal{R}_{\mathsf{ZisK}} \Leftrightarrow \begin{cases} 1 \leq |L| < 2^B, \\ \mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i) \text{ for every } i \in [|L|], \\ \mathrm{Balanced}(L). \end{cases}$$

The family $\mathcal{C}$, the local predicates and B are implicit in the notation.

The relation is invariant under permutations of L; it requires neither that every type occur nor that witnesses be distinct. Its three conjuncts are exactly what the recursion adds to the individual AIR proofs: a bound on the number of instances, one common public statement, and global bus balancing. Everything else that a ZisK proof establishes about the guest execution is a consequence of Local and $\mathrm{Balanced}$ for the specific AIRs of the circuit and is outside this section.

Definition 2.7 (Final argument system). The *final argument system* of ZisK is the non-interactive argument system Π^* for $\mathcal{R}_{\mathsf{ZisK}}$ whose proofs are Final proofs and whose verifier is

$$\Pi^*.\ \mathsf{Verify}(\boldsymbol{p}, \pi_{\mathrm{FIN}}) := V_{\mathrm{FINAL}(\boldsymbol{p}, \pi_{\mathrm{FIN}})}.$$

Its prover is the honest proving pipeline (execute, split, prove every instance, aggregate) and plays no role below.[1]

Unlike the reference architecture, where the external verifier first commits to the boundary states, Π^* coincides with Π_{FINAL} as an algorithm: the Final statement is exactly the external public statement $\boldsymbol{p}$. The two systems differ only in the relation against which they are judged.

Definition 2.8 (Correct-witness extractability). ZisK is *correct-witness extractable* (CWE) if Π^* is knowledge-sound for $\mathcal{R}_{\mathsf{ZisK}}$ in the sense of Definition 2.1. We write

$$\mathsf{Adv}^{\mathsf{cwe}}_{\mathsf{ZisK}}(\mathcal{A}) := \mathsf{Adv}^{\mathsf{ks}}_{\Pi^*}(\mathcal{A})$$

for the *CWE advantage* of a PPT adversary $\mathcal{A}$ and $\mathcal{E}_{\mathsf{ZisK}}$ for the extractor whose existence is asserted; the running-time convention of Definition 2.1 applies to it unchanged.

Remark 2.2 (Unrolling the definition). *Written out, Definition 2.8 says that there is a PPT extractor $\mathcal{E}_{\mathsf{ZisK}}$ such that for every PPT adversary $\mathcal{A}$ the following experiment returns 1 with negligible probability.*

1. ***Adversary.*** *$\mathcal{A}(1^\lambda)$ outputs a public statement $\boldsymbol{p}$ and a proof π_{FIN}.*
2. ***Verification.*** *If $V_{\mathrm{FINAL}(\boldsymbol{p}, \pi_{\mathrm{FIN}})} \neq \mathrm{accept}$, return 0.*
3. ***Extraction.*** *Run $\mathcal{E}_{\mathsf{ZisK}}(\boldsymbol{p}, \pi_{\mathrm{FIN}})$ to obtain a typed witness collection $L = \big((\tau_1, w_1), ..., \big(\tau_{|L|}, w_{|L|}\big)\big)$.*
4. ***Winning condition.*** *Return 1 if and only if $(\boldsymbol{p}; L) \notin \mathcal{R}_{\mathsf{ZisK}}$, i.e. if $|L| = 0$ or $|L| \geq 2^B$, or $\mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i)$ fails for some i, or some bus is unbalanced.*

This is Definition 2.1 instantiated at $(\mathcal{R}_{\mathsf{ZisK}}, \Pi^*)$ verbatim. The statement carries only $\boldsymbol{p}$, so, in contrast with the reference architecture, succinctness of Π^* and of Π_{FINAL} coincide.

2.3. From the Root Check to Bus Balancing.

The Final Verifier checks a single field equation, $s = 0$, on a value accumulated from the leaves under a challenge that the tree itself certifies to be derived from the leaves' commitments. By additivity of the fingerprint that value is $\mathcal{F}_\chi(\mathcal{M}_L)$ for the extracted collection L, so the check is a fingerprint zero test on the signed multiplicity of L, whereas the target relation demands $\mathcal{M}_L = 0$. This section bridges the gap. We first record that LogUp is a bus fingerprint in the sense of Definition 1.5, then that a zero test at a random challenge certifies balance, and finally that the challenge the tree certifies is random enough.

Lemma 2.9 (LogUp fingerprint). *$\mathcal{F}^{\mathrm{LogUp}}$ is a bus fingerprint with*

[1] Knowledge soundness depends only on the verification algorithm.

$$\varepsilon_{\mathcal{F}}(\Lambda) = \frac{\ell \cdot \binom{\Lambda}{2} + \Lambda - 1}{|\mathbb{K}|}.$$

Proof. Additivity. $\mathcal{F}_\chi^{\mathrm{LogUp}}(0) = 0$ is the empty sum. If $\mathcal{F}_\chi^{\mathrm{LogUp}}(D)$ and $\mathcal{F}_\chi^{\mathrm{LogUp}}(D')$ are both defined, no message of $\operatorname{supp} D \cup \operatorname{supp} D'$ is a pole; the support of $D + D'$ is contained in that union, and the sum defining $\mathcal{F}_\chi^{\mathrm{LogUp}}(D + D')$ splits message by message.

Soundness. Fix $D \neq 0$ with $|D| \leq \Lambda$, let $\chi = (\alpha, \gamma)$ be uniform in $\mathbb{K}^2$, and write $z_\gamma(b, v) := \hat{e}_\gamma(b, v)$ for $(b, v) \in \operatorname{supp} D$.

Compression collisions. Let E_γ be the event that $z_\gamma(b, v) = z_\gamma(b', v')$ for two distinct messages in $\operatorname{supp} D$. For a fixed pair of messages the difference is a polynomial in γ of degree at most ℓ, and it is nonzero: if $v \neq v'$ some coefficient of a positive power of γ is nonzero, and if $v = v'$ then $b \neq b'$ and the constant term is nonzero. A uniform γ is a root with probability at most $\ell/|\mathbb{K}|$, so $\Pr\big[E_\gamma\big] \leq \binom{\Lambda}{2} \cdot \ell/|\mathbb{K}|$.

Zero test. Fix γ with $\neg E_\gamma$, so the values $z_\gamma(b, v)$ are pairwise distinct, and define

$$P(X) := \sum_{(b,v) \in \operatorname{supp} D} D(b, v) \cdot \prod_{(b',v') \in \operatorname{supp} D \setminus \{(b,v)\}} \big(X + z_\gamma(b', v')\big) \in \mathbb{K}[X].$$

For any $(b_0, v_0) \in \operatorname{supp} D$,

$$P\big(-z_\gamma(b_0, v_0)\big) = D(b_0, v_0) \cdot \prod_{(b',v') \neq (b_0,v_0)} \big(z_\gamma(b', v') - z_\gamma(b_0, v_0)\big) \neq 0$$

because all factors are nonzero. Hence $P \neq 0$ and $\deg P \leq |D| - 1 \leq \Lambda - 1$. If $\mathcal{F}_\chi^{\mathrm{LogUp}}(D) = 0$, then all denominators are nonzero and multiplying through by their product gives $P(\alpha) = 0$, so α is a root of P. As α is uniform and independent of γ, this has probability at most $(\Lambda - 1)/|\mathbb{K}|$. A union bound over the two events gives the claim, and the bound is non-decreasing in Λ. □

Lemma 2.10 (Zero test). *Let $\mathcal{F}$ be a bus fingerprint, let L be a typed witness collection with $\neg\, \mathrm{Balanced}(L)$, and let χ be uniform in $\mathbb{K}^2$. Then*

$$\Pr_\chi \left[\sum_{i=1}^{|L|} \mathrm{BusCont}_\chi\big(\mathcal{I}_{\tau_i}, w_i\big) = 0 \right] \leq \varepsilon_{\mathcal{F}}(\Lambda(L)).$$

Proof. If the sum is 0, every summand $\mathcal{F}_\chi\big(\mathrm{Msg}_{\tau_i}(w_i)\big)$ is defined, so by additivity the sum equals $\mathcal{F}_\chi\big(\sum_i \mathrm{Msg}_{\tau_i}(w_i)\big) = \mathcal{F}_\chi(\mathcal{M}_L)$. Since L is unbalanced, $\mathcal{M}_L \neq 0$, and $|\mathcal{M}_L| \leq \Lambda(L)$ (Definition 1.7), so the soundness of $\mathcal{F}$ bounds the probability by $\varepsilon_{\mathcal{F}}(\Lambda(L))$. □

The zero test is exactly where the field-valued definition of $\mathrm{Balanced}$ enters: it certifies that every signed multiplicity vanishes in $\mathbb{F}$, no more. We now define the

game that captures what the root checks certify about an extracted collection, and bound it.

Definition 2.11 (Bus-argument soundness game). For an adversary $\mathcal{A}$ with access to the random oracles, the experiment $\mathsf{Exp}^{\mathsf{bus}}_{\mathcal{A}}(\lambda)$ proceeds as follows.

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs a typed witness collection $L = ((\tau_i, w_i))_{i\in[|L|]}$ and commitments $(\mathrm{com}_i)_{i\in[|L|]}$.
2. **Derivation.** Set $\boldsymbol{c} := \sum_{i=1}^{|L|} h(\tau_i, \mathrm{com}_i) \in \mathbb{Z}_p^n$ and $\chi := \mathsf{DeriveAlpha}(\boldsymbol{c})$.
3. **Winning condition.** Return 1 if and only if $1 \le |L| < 2^B$; $\mathsf{Commit}(w_i) = \mathrm{com}_i$ for every i; $\sum_{i=1}^{|L|} \mathrm{BusCont}_\chi\big(\mathcal{I}_{\tau_i}, w_i\big) = 0$; and $\neg\ \mathrm{Balanced}(L)$.

We write $\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}) := \Pr\big[\mathsf{Exp}^{\mathsf{bus}}_{\mathcal{A}}(\lambda) = 1\big]$.

Substituting $s_i := \mathrm{BusCont}_\chi\big(\mathcal{I}_{\tau_i}, w_i\big)$ and $\boldsymbol{c}_i := h(\tau_i, \mathrm{com}_i)$ shows that the winning condition is precisely "the root checks $s = 0$ and $\chi = \mathsf{DeriveAlpha}(\boldsymbol{c})$ pass on values honestly computed from L, yet L is unbalanced". This is the form in which the game is invoked in the proof of Theorem 2.21.

Proposition 2.12 (Bus soundness). *Let $\mathcal{F}$ be the bus fingerprint of the system. Under the idealizations of Remark 2.1, for every PPT adversary $\mathcal{A}$ making at most Q random-oracle queries there are reduction adversaries $\mathcal{D}^{\mathrm{cr}}$ and $\mathcal{D}^{\mathrm{ext}}$, each running $\mathcal{A}$ once in time $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, such that*

$$\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}) \le (Q+1)\cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{cr}}) + \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}}) + \frac{Q+2^B}{p^n}.$$

Proof. Let $\big(L, (\mathrm{com}_i)_i\big)$ be $\mathcal{A}$'s output and $\boldsymbol{c}$, χ as in the game. Number the $\mathsf{DeriveAlpha}$ queries in the order they are made and let $t^* \in [Q+1]$ be the index of the first query at $\boldsymbol{c}$, with $t^* := Q+1$ if $\mathcal{A}$ never queried $\boldsymbol{c}$; in the latter case the game's own derivation plays the role of a final, fresh query. For $t \in [Q+1]$ write $\boldsymbol{c}_t$ for the t-th query point, χ_t for its answer and $\mathcal{T}_{<t}$ for the transcript of all oracle queries and answers made before it; a typed commitment is *known at time t* if it was queried to h before the t-th $\mathsf{DeriveAlpha}$ query. We define bad events, bound each by a reduction or a counting argument, and show that off all of them the adversary cannot win.

Bad events.

- E_{fresh}: some pair (τ_i, com_i) of the output is not known at time t^*. Order the h-evaluations of the output pairs in time, counting the game's own evaluations last, and consider the last pair to receive its value g. At that moment $\boldsymbol{c}$ has been fixed (since time t^*) and so have the values of all other pairs, so $\boldsymbol{c} = \sum_i h(\tau_i, \mathrm{com}_i)$ forces $\mu \cdot g$ to equal a fixed target, where $\mu \in [1, 2^B)$ is the multiplicity of the pair in the collection and is invertible modulo p because $2^B < p$. A fresh oracle output hits the required value

with probability p^{-n}, and there are at most $Q + 2^B$ candidates for the last evaluation, so $\Pr[E_{\text{fresh}}] \le (Q + 2^B)/p^n$.

- E_{ext}: $\neg E_{\text{fresh}}$ and $\mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{<t^*}, \text{com}_i) \neq w_i$ for some i. Off E_{fresh} the commitment com_i was announced (queried to h) before time t^*, so this is the failure event of Definition 2.2 for the adversary $\mathcal{D}^{\text{ext}}$ that runs $\mathcal{A}$, announces com_i at its first h-query, and outputs (com_i, w_i). Hence $\Pr[E_{\text{ext}}] \le \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\text{ext}})$.
- E_{uniq}: at time t^* there are two distinct multisets $\mathcal{S} \neq \mathcal{S}'$ of typed commitments known at time t^*, both of cardinality below 2^B, with $\mathcal{H}(\mathcal{S}) = \mathcal{H}(\mathcal{S}') = \boldsymbol{c}$. The reduction $\mathcal{D}^{\text{cr}}$ runs $\mathcal{A}$ and outputs $(\mathcal{S}, \mathcal{S}')$, so $\Pr\left[E_{\text{uniq}}\right] \le \mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}(\mathcal{D}^{\text{cr}})$. Exhibiting the second multiset is the step covered by the second idealization of Remark 2.1: the pair is determined by the transcript, but a straight-line reduction cannot in general compute it for an algebraic accumulator (Remark 2.3).
- $E^{(t)}_{\chi}$ for $t \in [Q+1]$: define the *candidate collection* L^*_t as follows. If there is a unique multiset $\mathcal{S}_t$ of typed commitments known at time t, of cardinality below 2^B, with $\mathcal{H}(\mathcal{S}_t) = \boldsymbol{c}_t$, let $L^*_t := \left((\tau, \mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{<t}, \text{com}))\right)_{(\tau,\ \text{com}) \in \mathcal{S}_t}$ in a canonical order, discarding the collection if some extraction returns $\perp$; otherwise $L^*_t := \perp$. The event $E^{(t)}_{\chi}$ is that $L^*_t \neq \perp$, $\neg\ \text{Balanced}(L^*_t)$ and $\sum_i \text{BusCont}_{\chi_t}\left(\mathcal{I}_{\tau_i}, w_i\right) = 0$ over the entries of L^*_t. The candidate is a function of $\mathcal{T}_{<t}$ alone, whereas χ_t is uniform in $\mathbb{K}^2$ and independent of $\mathcal{T}_{<t}$; since $|L^*_t| < 2^B$ implies $\Lambda(L^*_t) \le \Lambda_{\max}$, Lemma 2.10 gives $\Pr\left[E^{(t)}_{\chi}\right] \le \varepsilon_{\mathcal{F}}(\Lambda_{\max})$.

Step A: off all events, the adversary does not win. Suppose $\mathcal{A}$ wins and none of $E_{\text{fresh}}, E_{\text{ext}}, E_{\text{uniq}}, E^{(t^*)}_{\chi}$ occurs. Off E_{fresh} every (τ_i, com_i) is known at time t^*, so the multiset $\mathcal{S} := \{(\tau_i, \text{com}_i)\}_i$ is a known multiset of cardinality $|L| < 2^B$ with $\mathcal{H}(\mathcal{S}) = \boldsymbol{c} = \boldsymbol{c}_{t^*}$; off E_{uniq} it is the unique such multiset, so $\mathcal{S}_{t^*} = \mathcal{S}$. Off E_{ext}, $\mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{<t^*}, \text{com}_i) = w_i$ for every i, so $L^*_{t^*}$ is a reordering of L; every quantity in the winning condition is invariant under reordering. The winning condition then says $\neg\ \text{Balanced}(L^*_{t^*})$ and $\sum_i \text{BusCont}_{\chi_{t^*}}\left(\mathcal{I}_{\tau_i}, w_i\right) = 0$ with $\chi = \chi_{t^*}$, which is $E^{(t^*)}_{\chi}$: a contradiction.

Conclusion. By Step A a win implies one of the bad events, and a union bound over $E_{\text{fresh}}, E_{\text{ext}}, E_{\text{uniq}}$ and $E^{(1)}_{\chi}, \ldots, E^{(Q+1)}_{\chi}$ gives the stated sum. Both reductions run $\mathcal{A}$ once and perform a scan of the transcript, so they run in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. □

Remark 2.3 (Removing the idealization). *Without the second idealization of Remark 2.1, the event E_{uniq} must be handled by rewinding. Run $\mathcal{A}$ a second time from its state just before the query t^*, answering it with a fresh χ'. If both runs win with the same accumulator and the extracted collections differ, the two multisets (or two openings of one commitment) form a collision for $\mathcal{H}$ or a break of Definition 2.2; if they coincide, a single unbalanced collection has passed the zero test at two*

independent challenges, which Lemma 2.10 bounds. The general forking lemma [BN06] then yields, schematically,

$$\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}) \le \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \sqrt{(Q+1)\big(\varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{cr}}) + \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}})\big)} + \frac{Q+1}{|\mathbb{K}|},$$

with reductions that run $\mathcal{A}$ twice. The square-root loss is inherent to rewinding at a Fiat–Shamir query [FS87] and is far from concretely tight. Making the linear bound of Proposition 2.12 rigorous is a proof obligation for the full security argument: either a knowledge-type assumption on LtHash (an efficient extractor that recovers the multiset from an accumulator the adversary produced), or an accumulator that is straight-line extractable in the ROM, at the cost of the homomorphism that makes independent aggregation possible.

Remark 2.4 (Field versus integer balance). *Proposition 2.12 delivers* $\mathrm{Balanced}(L)$ *with multiplicities in $\mathbb{F}$. To conclude that every message is sent and received the same number of times as integers one needs, in addition, that every multiplicity column of every AIR is range-checked to $[0, m_{\max}]$ (or to a symmetric interval with a fixed signed encoding) and that $\Lambda_{\max} \cdot m_{\max} < p$, so that vanishing in $\mathbb{F}$ lifts to vanishing in $\mathbb{Z}$. Whether the receiving sides of ZisK's lookup tables satisfy such a bound is a property of the individual AIRs and is recorded as an open item in Section 2.6.*

2.4. **Lemmas: Descending the Aggregation Tree.**

We state one extraction game and one lemma per layer of the aggregation tree, proceeding from the Final Verifier inward to the AIR arguments. Each lemma composes the straight-line extractors guaranteed by Definition 2.1.

Advantage notation. We collect the advantages introduced above: $\mathsf{Adv}^{\mathsf{ks}}_{\Pi}$ (knowledge-soundness error, Definition 2.1), $\mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}$ (Definition 2.2), $\mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}$ (Definition 2.3) and $\mathsf{Adv}^{\mathsf{bus}}$ (Definition 2.11). All advantages are functions of λ. In each lemma we exhibit explicit reduction adversaries $\mathcal{D}$ and state their running times; when a bound has the form $k \cdot \mathsf{Adv}^{\cdot}(\mathcal{D})$, the reduction $\mathcal{D}$ denotes the maximum-advantage adversary among the k per-node reductions constructed in the proof, each of which runs in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. Probabilities in the games below are over the coins of the adversary, the extractor and the random oracles; verification is deterministic.

2.4.1. *Final Layer.*

Definition 2.13 (Final extraction game). For an extractor $\mathcal{E}_{\mathrm{FIN}}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{fin}}_{(\mathcal{E}_{\mathrm{FIN}},\mathcal{A})}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^{\lambda})$ outputs a public statement $\boldsymbol{p} \in \mathcal{P}$ and a proof π_{FIN}.
2. **Filter.** If $V_{\mathrm{FINAL}(\boldsymbol{p},\pi_{\mathrm{FIN}})} \neq \mathrm{accept}$, return 0.
3. **Extraction.** Run $(\chi, \sigma, \pi) \leftarrow \mathcal{E}_{\mathrm{FIN}(\boldsymbol{p},\pi_{\mathrm{FIN}})}$, with $\sigma = (s, \tau, \kappa, \boldsymbol{c})$.

4. **Winning condition.** Return 1 if and only if $(\boldsymbol{p};(\chi,\sigma,\pi)) \notin \mathcal{R}_{\mathrm{FINAL}}$, i.e. unless $\mathsf{Typed}(\tau,\kappa)$, $V_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p},\chi,\sigma,\mathsf{vk}_{\mathrm{AGG}}),\pi) = \mathrm{accept}$, $s = 0$ and $\chi = \mathsf{DeriveAlpha}(\boldsymbol{c})$.

We write $\mathsf{Adv}^{\mathsf{fin}}_{\mathcal{E}_{\mathrm{FIN}}}(\mathcal{A})$ for the probability that the experiment returns 1.

Lemma 2.14 (Final extraction). *Assume Π_{FINAL} is knowledge-sound and let $\mathcal{E}_{\mathrm{FIN}} := \mathcal{E}_{\Pi_{\mathrm{FINAL}}}$ be its extractor. For every PPT adversary $\mathcal{A}$ there is a reduction $\mathcal{D}_{\mathrm{FIN}}$ with $\mathsf{Time}(\mathcal{D}_{\mathrm{FIN}}) = \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ and*

$$\mathsf{Adv}^{\mathsf{fin}}_{\mathcal{E}_{\mathrm{FIN}}}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}}).$$

Proof. $\mathcal{D}_{\mathrm{FIN}}$ runs $\mathcal{A}(1^\lambda)$ and forwards $(\boldsymbol{p},\pi_{\mathrm{FIN}})$ to the knowledge-soundness challenger of Π_{FINAL}. The challenger applies the same filter and runs the same extractor, and Definition 2.13 returns 1 exactly when π_{FIN} verifies but $(\boldsymbol{p};(\chi,\sigma,\pi)) \notin \mathcal{R}_{\mathrm{FINAL}}$, which is the event that $\mathcal{D}_{\mathrm{FIN}}$ wins. Since Π^* and Π_{FINAL} are the same algorithm, this is the base of the composition and no additional work is required. □

2.4.2. *Aggregation Layer.*

The aggregation extractor unrolls the tree below the Final Verifier's child until it reaches leaf-verifier proofs. The counter κ of the child statement tells it in advance how many leaves to expect, and Remark 1.1 guarantees that it never has to descend further than $\kappa - 1$ aggregation nodes.

Definition 2.15 (Aggregation extraction game). For a counter $\kappa \in [1, 2^B)$, an extractor $\mathcal{E}_{\mathrm{REC}}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{agg}}_{(\kappa,\mathcal{E}_{\mathrm{REC}},\mathcal{A})}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs $(\boldsymbol{p},\chi,\sigma,\pi)$ with $\sigma = (s,\tau,\kappa,\boldsymbol{c})$.
2. **Filter.** If $\neg\ \mathsf{Typed}(\tau,\kappa)$ or $V_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p},\chi,\sigma,\mathsf{vk}_{\mathrm{AGG}}),\pi) \neq \mathrm{accept}$, return 0. (These are the conjuncts of $\mathcal{R}_{\mathrm{FINAL}}$ that concern the child.)
3. **Extraction.** Run $((\sigma_1,\pi_1),\ldots,(\sigma_\kappa,\pi_\kappa)) \leftarrow \mathcal{E}_{\mathrm{REC}(\boldsymbol{p},\chi,\sigma,\pi)}$, with $\sigma_i = (s_i,\tau_i,\kappa_i,\boldsymbol{c}_i)$.
4. **Winning condition.** Return 1 unless, for every $i \in [\kappa]$, $\tau_i \in [M]$, $\kappa_i = 1$ and $V_{\mathrm{LEAF}}^{(\tau_i)}((\boldsymbol{p},\chi,\sigma_i,\mathsf{vk}_{\mathrm{AGG}}),\pi_i) = \mathrm{accept}$, and moreover $\sum_{i=1}^{\kappa} s_i = s$ and $\sum_{i=1}^{\kappa} \boldsymbol{c}_i = \boldsymbol{c}$.

We write $\mathsf{Adv}^{\mathsf{agg}}_{(\kappa,\mathcal{E}_{\mathrm{REC}})}(\mathcal{A})$ for the probability that the experiment returns 1.

Lemma 2.16 (Aggregation tree unrolling). *Assume Π_{AGG} is knowledge-sound and let $\kappa \in [1, 2^B)$. There is an extractor $\mathcal{E}_{\mathrm{REC}}$, given in the proof, such that for every PPT adversary $\mathcal{A}$ there is a reduction $\mathcal{D}_{\mathrm{AGG}}$ running in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ with*

$$\mathsf{Adv}^{\mathsf{agg}}_{(\kappa,\mathcal{E}_{\mathrm{REC}})}(\mathcal{A}) \le (\kappa-1)\cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}}).$$

$\mathcal{E}_{\mathrm{REC}}$ makes at most $\kappa - 1$ calls to $\mathcal{E}_{\Pi_{\mathrm{AGG}}}$ and runs in $(\kappa-1)\cdot\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

Proof. The game returns 0 unless the filter passes, so we assume it does and bound the probability that extraction violates the winning condition. The argument runs in four steps.

Step 1: recursion tree and bad events. $\mathcal{E}_{\mathrm{REC}}$ unrolls a binary tree depth-first. A node v is a pair (x_v, π_v) with $x_v = (\boldsymbol{p}, \chi, \sigma_v, \mathsf{vk}_{\mathrm{AGG}})$; the root is $\mathcal{A}$'s output. If $\sigma_v.\tau \in [M]$, the node is a *leaf* and the extractor records (σ_v, π_v). If $\sigma_v.\tau = M+1$, the extractor runs $\mathcal{E}_{\Pi_{\mathrm{AGG}}}(x_v, \pi_v)$ to obtain a candidate witness $((\sigma_L, \pi_L), (\sigma_R, \pi_R))$ and recurses into the left child $((\boldsymbol{p}, \chi, \sigma_L, \mathsf{vk}_{\mathrm{AGG}}), \pi_L)$ and then the right child; if the candidate is not a valid $\mathcal{R}_{\mathrm{AGG}}$-witness, the extractor aborts. For $t \geq 1$ let

: the first $\mathcal{E}_{\Pi_{\mathrm{AGG}}}$ invocation (in unroll order) that does not return a valid $\mathcal{R}_{\mathrm{AGG}}$ -witness for its node's statement is invocation

The events are disjoint, and at every invocation the node's proof verifies under $V(\mathsf{vk}_{\mathrm{AGG}}, \cdot, \cdot)$: the root's by the filter (since $\mathsf{Typed}(\tau, \kappa)$ with $\tau = M+1$ and $V_{\mathrm{REC}}^{(M+1)}[\mathsf{vk}_{\mathrm{AGG}}] = V(\mathsf{vk}_{\mathrm{AGG}}, \cdot, \cdot)$), a child's by its parent's valid witness. Now count. Before the first failure, the $t-1$ valid aggregation nodes form a binary tree whose frontier consists of t nodes, each not yet expanded; by the counter invariant every frontier node carries an integer counter at least 1, and the counters of the frontier sum to κ. Hence $t \leq \kappa$. Moreover, a frontier node of type $M+1$ has counter at least 2 by Typed, so if $\kappa - 1$ valid nodes exist the frontier has κ nodes with counter exactly 1, all of type in $[M]$, and no further invocation exists. Only $B_1, ..., B_{\kappa-1}$ are possible, and off all of them the tree has exactly $\kappa - 1$ aggregation nodes and κ leaves; for $\kappa = 1$ the root is a leaf and there is nothing to extract.

Step 2: one reduction per invocation. $\mathcal{D}_{\mathrm{AGG}}^{(t)}$ runs $\mathcal{A}(1^\lambda)$, unrolls the tree up to the t-th invocation (which is deterministic given $\mathcal{A}$'s output and the earlier extractions) and forwards that node's (x_v, π_v) to the knowledge-soundness challenger of Π_{AGG}. It wins whenever B_t occurs, so $\Pr[B_t] \leq \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}\left(\mathcal{D}_{\mathrm{AGG}}^{(t)}\right)$, and it runs in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

Step 3: off the bad events, the extractor succeeds. Assume $\neg B_1 \wedge ... \wedge \neg B_{\kappa-1}$; every statement below is then deterministic. We show by strong induction on the counter κ_v that for every node v of the unrolled tree, the leaves recorded below v number exactly κ_v, each has a type in $[M]$ and counter 1, each verifies under $V_{\mathrm{LEAF}}^{(\tau_i)}$ on the statement $(\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\mathrm{AGG}})$, and their partial sums and challenge contributions add up to $\sigma_v.s$ and $\sigma_v.\boldsymbol{c}$.

- *Leaf nodes.* If $\sigma_v.\tau \in [M]$, then Typed (asserted by the parent's valid witness, or by the filter for the root) gives $\kappa_v = 1$, and the same source gives $V_{\mathrm{REC}}^{(\tau_v)}[\mathsf{vk}_{\mathrm{AGG}}](x_v, \pi_v) = V_{\mathrm{LEAF}}^{(\tau_v)}(x_v, \pi_v) = \mathrm{accept}$. The sums are trivial.
- *Aggregation nodes.* If $\sigma_v.\tau = M+1$, the valid witness returned for v provides typed children with $\kappa_L + \kappa_R = \kappa_v$ over the integers and $\kappa_X \in [1, \kappa_v - 1]$, each verifying under $V_{\mathrm{REC}}^{(\tau_X)}[\mathsf{vk}_{\mathrm{AGG}}]$ on a statement that, by the propagation invariant, carries the same $(\boldsymbol{p}, \chi, \mathsf{vk}_{\mathrm{AGG}})$; a child of type

$M+1$ is therefore verified by $V(\mathsf{vk}_{\mathrm{AGG}},\cdot,\cdot) = V_{\mathrm{AGG}}$ and is itself an aggregation node of the tree. The induction hypothesis applies to both children; concatenating their leaf lists gives $\kappa_L + \kappa_R = \kappa_v$ leaves, and $s_v = s_L + s_R$, $\boldsymbol{c}_v = \boldsymbol{c}_L + \boldsymbol{c}_R$ make the sums telescope.

Applied to the root, this is exactly the winning condition of Definition 2.15.

Step 4: conclusion. Off all bad events the extractor meets the winning condition, so the game returns 1 only if some B_t occurs, and only $B_1, \ldots, B_{\kappa-1}$ are possible. By a union bound

$$\mathsf{Adv}^{\mathsf{agg}}_{(\kappa,\mathcal{E}_{\mathrm{REC}})}(\mathcal{A}) \le \sum_{t=1}^{\kappa-1} \Pr[B_t] \le (\kappa-1)\cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}}),$$

with $\mathcal{D}_{\mathrm{AGG}}$ the maximum-advantage reduction among the $\mathcal{D}^{(t)}_{\mathrm{AGG}}$. The extractor invokes $\mathcal{E}_{\Pi_{\mathrm{AGG}}}$ once per aggregation node, at most $\kappa-1$ times, each call costing at most $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. □

The proof relies on the typing predicate at three places where a proof could *verify* for a statement outside the relation's support, which knowledge soundness does not exclude: a child of type $M+1$ with counter 1 (whose statement has no witness), a child of type in $[M]$ with a counter other than 1 (outside the support of the leaf relation), and a child type outside $[M+1]$ (for which no verifier is selected). In each case the parent's valid witness excludes the child, so the offending proof is charged to the parent's invocation. This is the role played by the step-count typing in the reference architecture's combine layer.

2.4.3. *Leaf-Verifier Layer.*

Definition 2.17 (Leaf-verifier extraction game). For $\tau \in [M]$, an extractor $\mathcal{E}^{(\tau)}_{\mathrm{LEAF}}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{leaf}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{LEAF}},\mathcal{A}\right)}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs $(\boldsymbol{p},\chi,\sigma,\mathsf{vk})$ with $\sigma = (s,\tau',\kappa,\boldsymbol{c})$, and a proof π_{LEAF}.
2. **Filter.** If $V^{(\tau)}_{\mathrm{LEAF}}((\boldsymbol{p},\chi,\sigma,\mathsf{vk}),\pi_{\mathrm{LEAF}}) \neq \mathrm{accept}$, return 0.
3. **Extraction.** Run $(\mathrm{com},\pi_{\mathrm{AIR}}) \leftarrow \mathcal{E}^{(\tau)}_{\mathrm{LEAF}}((\boldsymbol{p},\chi,\sigma,\mathsf{vk}),\pi_{\mathrm{LEAF}})$.
4. **Winning condition.** Return 1 if and only if $((\boldsymbol{p},\chi,\sigma,\mathsf{vk});(\mathrm{com},\pi_{\mathrm{AIR}})) \notin \mathcal{R}^{(\tau)}_{\mathrm{LEAF}}$, i.e. unless $\tau' = \tau$, $\kappa = 1$, $V^{(\tau)}_{\mathrm{AIR}}((\boldsymbol{p},\chi,s,\mathrm{com}),\pi_{\mathrm{AIR}}) = \mathrm{accept}$ and $\boldsymbol{c} = h(\tau,\mathrm{com})$.

We write $\mathsf{Adv}^{\mathsf{leaf}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{LEAF}}\right)}(\mathcal{A})$ for the probability that the experiment returns 1.

Lemma 2.18 (Leaf-verifier extraction). *Assume $\Pi^{(\tau)}_{\mathrm{LEAF}}$ is knowledge-sound and let $\mathcal{E}^{(\tau)}_{\mathrm{LEAF}} := \mathcal{E}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}$. For every PPT adversary $\mathcal{A}$,*

$$\mathsf{Adv}^{\mathsf{leaf}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{LEAF}}\right)}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{LEAF}}\right), \quad \mathcal{D}^{(\tau)}_{\mathrm{LEAF}} := \mathcal{A}.$$

Proof. The experiment is the knowledge-soundness game of $\Pi^{(\tau)}_{\mathrm{LEAF}}$: same filter, same extractor, and the winning condition is membership of the extracted pair in $\mathcal{R}^{(\tau)}_{\mathrm{LEAF}}$. A statement with $\tau' \neq \tau$ or $\kappa \neq 1$ lies outside the support of the relation; a verifying proof for it is a knowledge-soundness break, and the experiment returns 1 for it as well, so the two advantages coincide. The key vk is not constrained at this layer and is only forwarded. □

The type under which a leaf is extracted is the type $\sigma_i.\tau$ read from its statement, which is also the type that selected the verifier in the aggregation layer. The extracted AIR proof is therefore a proof for $\mathcal{A}_{\sigma_i.\tau}$, and this is what makes the output of the composed extractor a *typed* collection.

2.4.4. *AIR Layer.*

Definition 2.19 (AIR extraction game). For $\tau \in [M]$, an extractor $\mathcal{E}^{(\tau)}_{\mathrm{AIR}}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{air}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{AIR}},\mathcal{A}\right)}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs a statement $x = (\boldsymbol{p}, \chi, s, \mathrm{com})$ and a proof π_{AIR}.
2. **Filter.** If $V^{(\tau)}_{\mathrm{AIR}}(x, \pi_{\mathrm{AIR}}) \neq \mathrm{accept}$, return 0.
3. **Extraction.** Run $(\boldsymbol{T}, \boldsymbol{a}) \leftarrow \mathcal{E}^{(\tau)}_{\mathrm{AIR}}(x, \pi_{\mathrm{AIR}})$.
4. **Winning condition.** Return 1 if and only if $(x; (\boldsymbol{T}, \boldsymbol{a})) \notin \mathcal{R}^{(\tau)}_{\mathrm{AIR}}$, i.e. unless $\mathrm{Local}_\tau(\boldsymbol{p}, (\boldsymbol{T}, \boldsymbol{a}))$, $s = \mathrm{BusCont}_\chi(\mathcal{I}_\tau, (\boldsymbol{T}, \boldsymbol{a}))$ and $\mathsf{Commit}(\boldsymbol{T}, \boldsymbol{a}) = \mathrm{com}$.

We write $\mathsf{Adv}^{\mathsf{air}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{AIR}}\right)}(\mathcal{A})$ for the probability that the experiment returns 1.

Lemma 2.20 (AIR extraction). *Assume $\Pi^{(\tau)}_{\mathrm{AIR}}$ is knowledge-sound and let $\mathcal{E}^{(\tau)}_{\mathrm{AIR}} := \mathcal{E}_{\Pi^{(\tau)}_{\mathrm{AIR}}}$. For every PPT adversary $\mathcal{A}$,*

$$\mathsf{Adv}^{\mathsf{air}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{AIR}}\right)}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{AIR}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{AIR}}\right), \quad \mathcal{D}^{(\tau)}_{\mathrm{AIR}} := \mathcal{A},$$

and whenever the experiment returns 0 after passing the filter, the extracted witness $(\boldsymbol{T}, \boldsymbol{a}) \in \mathcal{W}_\tau$ satisfies $\mathrm{Local}_\tau(\boldsymbol{p}, (\boldsymbol{T}, \boldsymbol{a}))$, has bus contribution s under χ, and commits to com.

Proof. The experiment is the knowledge-soundness game of $\Pi^{(\tau)}_{\mathrm{AIR}}$, and the structure of the extracted witness is the definition of $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$. □

2.5. **Main Theorem.**

Theorem 2.21 (Correct-witness extractability). *Let ZisK be a ZisK proof system (Definition 2.5) and let $\kappa_{\max} := 2^B - 1$. Under the idealizations of Remark 2.1, for every PPT adversary $\mathcal{A}$ making at most Q random-oracle queries, with CWE advantage $\varepsilon := \mathsf{Adv}^{\mathsf{cwe}}_{\mathsf{ZisK}}(\mathcal{A})$, there exist PPT reduction adversaries $\mathcal{D}_{\mathrm{FIN}}$, $\mathcal{D}_{\mathrm{AGG}}$, $\left\{\mathcal{D}^{(\tau)}_{\mathrm{LEAF}}\right\}_\tau$, $\left\{\mathcal{D}^{(\tau)}_{\mathrm{AIR}}\right\}_\tau$, $\mathcal{D}^{\mathrm{cr}}$ and $\mathcal{D}^{\mathrm{ext}}$ such that*

$$\varepsilon \le \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}}) + (\kappa_{\max}-1)\cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}})$$
$$+\kappa_{\max}\cdot \max_{\tau\in[M]} \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{LEAF}}\right) + \kappa_{\max}\cdot \max_{\tau\in[M]} \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{AIR}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{AIR}}\right)$$
$$+(Q+1)\cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{cr}}) + \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}}) + \frac{Q+2^B}{p^n}.$$

All reduction adversaries invoke $\mathcal{A}$ exactly once and run in time at most $(3\kappa_{\max}+1)\cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. In particular, if every advantage on the right-hand side is negligible in λ, then ZisK is correct-witness extractable (Definition 2.8).

Proof. Let $\mathcal{A}$ be a PPT adversary with CWE advantage ε; we bound ε. We construct $\mathcal{E}_{\mathsf{ZisK}}$ by composing the sub-extractors of Lemma 2.14, Lemma 2.16, Lemma 2.18 and Lemma 2.20, each straight-line and each applied to the values extracted by the preceding layer. Let $(\boldsymbol{p}, \pi_{\mathrm{FIN}})$ be $\mathcal{A}$'s output and assume $V_{\mathrm{FINAL}(\boldsymbol{p},\pi_{\mathrm{FIN}})} = \mathrm{accept}$, since otherwise $\mathcal{A}$ does not win.

Step 1 (Final layer). Apply $\mathcal{E}_{\mathrm{FIN}}$ to $(\boldsymbol{p}, \pi_{\mathrm{FIN}})$ to obtain (χ, σ, π) with $\sigma = (s, \tau, \kappa, \boldsymbol{c})$ such that $\mathsf{Typed}(\tau, \kappa)$, $V^{(\tau)}_{\mathrm{REC}}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}_{\mathrm{AGG}}), \pi) = \mathrm{accept}$, $s = 0$ and $\chi = \mathsf{DeriveAlpha}(\boldsymbol{c})$. This fails with probability at most $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}})$ by Lemma 2.14. In particular $\kappa \in [1, 2^B)$, so $\kappa \le \kappa_{\max}$.[2]

Step 2 (Aggregation layer). Apply $\mathcal{E}_{\mathrm{REC}}$ to $(\boldsymbol{p}, \chi, \sigma, \pi)$; its filter is satisfied by Step 1. Obtain $((\sigma_1, \pi_1), ..., (\sigma_\kappa, \pi_\kappa))$ with $\sigma_i = (s_i, \tau_i, 1, \boldsymbol{c}_i)$, $\tau_i \in [M]$, $V^{(\tau_i)}_{\mathrm{LEAF}}((\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\mathrm{AGG}}), \pi_i) = \mathrm{accept}$, $\sum_i s_i = s = 0$ and $\sum_i \boldsymbol{c}_i = \boldsymbol{c}$. This fails with probability at most $(\kappa-1)\cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}})$ by Lemma 2.16, and the extractor runs in $(\kappa-1)\cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

Step 3 (Leaf-verifier layer, applied $\boldsymbol{\kappa}$ times). For each $i \in [\kappa]$ apply $\mathcal{E}^{(\tau_i)}_{\mathrm{LEAF}}$ to $((\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\mathrm{AGG}}), \pi_i)$ to obtain $\left(\mathrm{com}_i, \pi^{\mathrm{AIR}}_i\right)$ with $V^{(\tau_i)}_{\mathrm{AIR}}\left((\boldsymbol{p}, \chi, s_i, \mathrm{com}_i), \pi^{\mathrm{AIR}}_i\right) = \mathrm{accept}$ and $\boldsymbol{c}_i = h(\tau_i, \mathrm{com}_i)$. Each invocation fails with probability at most $\mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau_i)}_{\mathrm{LEAF}}}\left(\mathcal{D}^{(\tau_i)}_{\mathrm{LEAF}}\right)$ by Lemma 2.18; the total over κ calls is at most $\kappa \cdot \max_\tau \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{LEAF}}\right)$.

Step 4 (AIR layer, applied $\boldsymbol{\kappa}$ times). For each $i \in [\kappa]$ apply $\mathcal{E}^{(\tau_i)}_{\mathrm{AIR}}$ to $\left((\boldsymbol{p}, \chi, s_i, \mathrm{com}_i), \pi^{\mathrm{AIR}}_i\right)$ to obtain $w_i = (\boldsymbol{T}_i, \boldsymbol{a}_i) \in \mathcal{W}_{\tau_i}$ with $\mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i)$, $s_i = \mathrm{BusCont}_\chi\left(\mathcal{I}_{\tau_i}, w_i\right)$ and $\mathsf{Commit}(w_i) = \mathrm{com}_i$. The total failure probability over κ calls is at most $\kappa \cdot \max_\tau \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{AIR}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{AIR}}\right)$ by Lemma 2.20.

Step 5 (Assembly). Output $L := ((\tau_1, w_1), ..., (\tau_\kappa, w_\kappa))$. Deterministically, given that Steps 1 to 4 succeeded: $|L| = \kappa \in [1, 2^B)$; $\mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i)$ for every i; $\boldsymbol{c} = \sum_i h(\tau_i, \mathrm{com}_i) = \sum_i h(\tau_i, \mathsf{Commit}(w_i))$; $\chi = \mathsf{DeriveAlpha}(\boldsymbol{c})$; and $\sum_i \mathrm{BusCont}_\chi\left(\mathcal{I}_{\tau_i}, w_i\right) = \sum_i s_i = s = 0$.

[2] This composition is where the first idealization of Remark 1 is invoked: a straight-line extractor for a succinct Π_{FINAL} does not exist in the plain model, and composing it with the extractors below requires relativized arguments.

Step 6 (Bus soundness). Let $\mathcal{A}_{\mathrm{bus}}$ be the algorithm that runs $\mathcal{A}$ and Steps 1 to 5 and outputs $\big(L, (\mathrm{com}_i)_i\big)$; it makes the same oracle queries as $\mathcal{A}$ plus those of the extractors. When Steps 1 to 4 succeed, Step 5 shows that the output satisfies every conjunct of the winning condition of Definition 2.11 except possibly $\neg$ Balanced(L), and the game's derivation reproduces the extracted $\boldsymbol{c}$ and χ. Hence

$$\Pr[\text{Steps 1 to 4 succeed} \wedge \neg \text{ Balanced}(L)] \le \mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}_{\mathrm{bus}})$$
$$\le (Q+1)\cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{cr}}) + \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}}) + \frac{Q+2^B}{p^n}$$

by Proposition 2.12, where $\mathcal{D}^{\mathrm{cr}}$ and $\mathcal{D}^{\mathrm{ext}}$ run $\mathcal{A}_{\mathrm{bus}}$ once.

Conclusion. If Steps 1 to 4 succeed and L is balanced, then $(\boldsymbol{p}; L) \in \mathcal{R}_{\mathsf{ZisK}}$ by Step 5 and Definition 2.6, so $\mathcal{A}$ does not win. Summing the failure probabilities,

$$\begin{aligned}
\varepsilon \le\ & \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}}) && \text{(Step 1)}\\
& +(\kappa-1)\cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}}) && \text{(Step 2)}\\
& +\kappa\cdot \max_{\tau} \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{LEAF}}\right) && \text{(Step 3)}\\
& +\kappa\cdot \max_{\tau} \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{AIR}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{AIR}}\right) && \text{(Step 4)}\\
& +\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}_{\mathrm{bus}}) && \text{(Step 6)},
\end{aligned}$$

and replacing κ by $\kappa_{\max}$, which only increases the right-hand side, gives the stated bound. All coefficients are polynomially bounded, so the right-hand side is negligible whenever each advantage is. Unlike the step count of the reference architecture, κ is not a system parameter but part of the extracted witness; the bound with κ is therefore a per-tree refinement and the bound with $\kappa_{\max}$ the unconditional one.

Running time. $\mathcal{E}_{\mathsf{ZisK}}$ never runs $\mathcal{A}$; its cost is the straight-line sub-extractor calls plus deterministic work: one call to $\mathcal{E}_{\Pi_{\mathrm{FINAL}}}$, at most $\kappa-1$ calls to $\mathcal{E}_{\Pi_{\mathrm{AGG}}}$, and κ calls each to the leaf-verifier and AIR extractors, i.e. at most 3κ calls in total. Under the convention that each call costs at most $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, $\mathsf{Time}(\mathcal{E}_{\mathsf{ZisK}}) \le 3\kappa_{\max}\cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. Each reduction adversary runs $\mathcal{A}$ once and re-runs the extraction chain above its own layer, hence runs in at most $(3\kappa_{\max}+1)\cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. (Strictly, the convention of Definition 2.1 bounds each extractor by the time of the composed adversary that produced its input, which compounds along the recursion to $O(\kappa_{\max}^2)\cdot \mathsf{Time}(\mathcal{A})$; the linear figure uses the per-call convention and is part of the idealization of Remark 2.1.) In particular $\mathcal{E}_{\mathsf{ZisK}}$ is PPT whenever $\mathcal{A}$ is, since $2^B \le \mathsf{poly}(\lambda)$. □

2.6. **Discussion.**

Every soundness, binding or extractability term that appears in Theorem 2.21 is either defined in this document, inherited from the backend, or an explicit

idealization. Table 2 records the status of each, together with the obligations that a full security argument must discharge.

Term	Status	Where / what remains
$\mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{AIR}}}$	backend assumption	Knowledge soundness of each STARK argument for $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$ in commit-and-prove form (Definition 1.9); quantified by the backend's own analysis.
$\mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}$, $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}$, $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}$	backend assumption	Knowledge soundness of the recursive STARK circuits for Definition 1.10, Definition 1.11, Definition 1.12, including the typing predicate and the vk-dispatch.
straight-line composition	idealization	Remark 2.1 (first item); relativized arguments do not exist [BCG24].
$\mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}$	defined (Definition 2.2)	Standard for Merkle trees over a random oracle.
$\mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}$	defined (Definition 2.3)	LtHash on multisets of cardinality below 2^B; requires a collision-resistant inner digest of at least 2λ bits.
accumulator uniqueness	idealization	Remark 2.1 (second item); rigorous alternatives in Remark 2.3.
$\varepsilon_{\mathcal{F}}$	defined (Definition 1.5); proved for LogUp (Lemma 2.9)	Statistical; requires the challenge pair to be a random-oracle output (Definition 2.4). The circuit proves a sub-relation of $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$ (Section 1.2).
$\mathrm{Balanced}(L)$	proved (Proposition 2.12)	Field-valued; integer balance needs multiplicity range checks (Remark 2.4).

TABLE 2. Definition and obligation ledger for the terms of Theorem 2.21.

The following items are outside the theorem and are recorded as obligations for the sections that build on it.

- **From witnesses to executions.** Theorem 2.21 yields a balanced collection of locally satisfying AIR witnesses for a common public statement. The semantic claim of ZisK, that such a collection encodes one execution of the committed program from the boot state to a halting state whose

public outputs are $\boldsymbol{p}$, requires the memory argument, the continuation stitching and the per-chip semantics, each with its own relation and lemma. These compose with Theorem 2.21 by strengthening Local and by reading off the meaning of $\mathrm{Balanced}$ for the specific buses.

- **Integer multiplicities.** See Remark 2.4: a bound on every multiplicity column, in particular on the receiving side of shared lookup tables, is needed to lift field balance to multiset equality.
- **The accumulator.** The linear bound of Proposition 2.12 rests on the second idealization of Remark 2.1. Discharging it (Remark 2.3) is the main cryptographic obligation left by this section; the bound that forking gives is valid but concretely loose.
- **Circuit requirements.** The relations of Section 1.4 require the circuits to enforce the typing predicate on children (Remark 1.1), to hash the type together with the commitment into the challenge contribution, and to expose the trace commitment as a public input of the AIR argument. Each is a small change relative to the whitepaper's description and each is used by the proofs.
- **Fixed and cached data.** Local_τ treats cached columns as fixed data bound through the verification key. The correctness of cached columns (for instance precomputed operation tables) and the binding of the program ROM are therefore part of the key ledger (Table 1) and of the setup, not of the recursion argument.
- **Concrete parameters.** With $B = 20$ the running-time factor $3\kappa_{\max}$ is about $3 \cdot 10^6$ and the coefficient of the recursive knowledge-soundness terms is about 10^6; both are polynomial but not small, and any concrete security claim must account for them together with the query bound Q and the caveats of Remark 2.1.
- **Completeness.** The honest pipeline produces, for every balanced collection of at most $2^B - 1$ locally satisfying witnesses, an accepting Final proof; we have not stated this formally, and a complete treatment should.

References

Hab22. Haböck, U.: Multivariate lookups based on logarithmic derivatives, `https://eprint.iacr.org/2022/1530`

Zis25. ZisK: Secure Challenge Derivation in ZisK, `https://hackmd.io/SXKiWASTRu2QYgy7B-NGmg`

Cla+03. Clarke, D.E., Devadas, S., Dijk, M. van, Gassend, B., Suh, G.E.: Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking. In: Laih, C.-S. (ed.) Advances in Cryptology – ASIACRYPT~2003. pp. 188–207. Springer, Berlin, Heidelberg (2003)

GKS23. Grassi, L., Khovratovich, D., Schofnegger, M.: Poseidon2: A Faster Version of the Poseidon Hash Function. In: El Mrabet, N., De Feo, L., and Duquesne, S. (eds.) AFRICACRYPT 23: 14th International Conference on Cryptology in Africa. pp. 177–203. Springer, Cham (2023)

BCS16. Ben-Sasson, E., Chiesa, A., Spooner, N.: Interactive Oracle Proofs. In: TCC 2016-B: 14th Theory of Cryptography Conference, Part II. pp. 31–60. Springer, Berlin, Heidelberg (2016)

BCG24. Barbara, A., Chiesa, A., Guan, Z.: Relativized Succinct Arguments in the ROM Do Not Exist, https://eprint.iacr.org/2024/728

Ajt96. Ajtai, M.: Generating Hard Instances of Lattice Problems (Extended Abstract). In: 28th Annual ACM Symposium on Theory of Computing. pp. 99–108. ACM Press (1996)

BN06. Bellare, M., Neven, G.: Multi-signatures in the plain public-Key model and a general forking lemma. In: ACM CCS 2006: 13th Conference on Computer and Communications Security. pp. 390–399. ACM Press (2006)

FS87. Fiat, A., Shamir, A.: How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In: Odlyzko, A.M. (ed.) Advances in Cryptology – CRYPTO'86. pp. 186–194. Springer, Berlin, Heidelberg (1987)